Tracking Monte Carlo Convergence in the Estimation of π Using Python

Adam Feldman[a], Luke Wilson[a]

[a]*Department of Physics, Miami University, 500 E. Spring Street, Oxford, OH, USA.*

1. The Problem

Any student studying mathematics will likely encounter the constant π early in their studies. Representing the ratio of any circle's circumference to its diameter, π is an irrational number, meaning nobody knows its true value. Due to π's irrational nature, mathematicians are forced to use estimation techniques in order to obtain an accurate value. The first sign of π's estimation comes from the Babylonians (3000 BC), who calculated the area of a circle by taking 3 times the square of its radius, giving a value of $\pi = 3$. Additionally, the pyramids of Cheops and Sneferu in Egypt show the estimation of π at $3\frac{1}{7}$. This estimate improved in 1850 BC, as documented in the Rhind Papyrus, to $\pi = (\frac{16}{9})^2$ [1]. The first calculation of π was done by Archimedes, who approximated the area of a circle by using the Pythagorean Theorem to find the areas of two regular polygons. The polygon inscribed within the circle and the polygon within which the circle was circumscribed. Since the actual area of the circle lies between the areas of the inscribed and circumscribed polygons, Archimedes knew that he had not found the value of π but only an approximation within those limits. Doing this, Archimedes showed that π is between $3\ \frac{1}{7}$ and $3\ \frac{10}{71}$. Zu Chongzhi, a brilliant Chinese mathematician and astronomer, utilized a similar approach, calculating the value of the ratio of the circumference of a circle to its diameter to be 355/113 [2]. While estimations continued to improve, modern computational methods are much easier to use and more accurate than these analytical solutions. One such method that we will continue to discuss in this paper is called the Monte Carlo Method.

2. Theoretical Background

Named after the famous gambling destination in Monaco, Monte Carlo Simulations are a way to model probabilities involving random variables. The Monte Carlo Method was invented by John von Neumann and Stanislaw Ulam during World War II to improve decision-making under uncertain conditions. Additionally, it has many applications in artificial intelligence, stock prices, sales forecasting, project management, and pricing [3]. Unlike other forecasting models, Monte Carlo Simulation predicts outcomes by leveraging a probability distribution for variables with inherent uncertainty. It then iterates the results over and over, each time using a different set of random numbers between the minimum and maximum values. In a typical Monte Carlo experiment, this exercise can be repeated thousands of times to produce a large number of likely outcomes [4]. In many models, outcomes follow a normal distribution, with the most likely return at the center. The advantage of using Monte Carlo Simulations is that they take a number of random variables and then average them, rather than starting out with an average, providing a more probable range of results. Of course, no simulation can pinpoint the exact outcome. The Monte Carlo method aims at a sounder estimate of the probability that an outcome will differ from a projection. The more simulations you do, the more accurate this normal distribution becomes. [3] This is called Monte Carlo Convergence, the process of improving the accuracy of a simulation through the use of increasing iterations.

3. Relevant Analysis

The Monte Carlo method utilizes random generation to approximate mathematical values through the use of repeated sampling. To approximate π, a sampling domain of a square, with side length 2 (from -1 to 1) is used, with an area of 4. The unit circle is then overlaid inside the square, with the area of π. This ratio between the circle and the square gives a result of $\pi/4$. Through the hallmark process of the Monte Carlo simulation technique, a large sample size of randomly generated points that satisfy the unit circle equation (equation 1) are created and then extracted as these pass the test and are including in the π estimation.

$$x^2 + y^2 \leq 1 \tag{1}$$

To complete the π estimation, equation 2 is used.

$$\pi \approx 4 \times \text{samples} \tag{2}$$

The Monte Carlo simulation convergence rate is $O\frac{1}{\sqrt{n}}$, where if you double the accuracy, a 4x sample increase is required [5]. This means that the method has a fundamental limit for high precision, where 5 digits requires millions of samples and 6 digits would require billions. At any step, the decimal digits of accuracy are defined by equation 3 [5].

$$d = -\log_{10}(|\text{estimate} - \pi|) \tag{3}$$

For all calculations, taking the median of multiple runs is required because individual runs are random, where the error can change drastically from run to run.

4. Computational approach

To create a computational approach that would allow for a Monte Carlo simulation approaching π, a multi-step process was used. First, a simple simulation was established containing the core formula used throughout the rest of the program. To start, 10,000 x and y values are uniformly generated between -1 and 1, to create a square of area 4. Then, based on equation 1, points inside the circle are isolated and multiplied by the ratio of 4 to give the estimate for π.

Next, a loop simulation was created using the same logic as the simple simulation setup, except repeated from a starting point of a single sample until the sample count hit a predetermined 1,000,000 samples. For each iteration, the sample count increased by a single order of magnitude. At each step, the estimate was printed to show the improvement per order of magnitude in the Monte Carlo simulation.

Through these combinations, a handful of custom functions were created. The first, `monte_carlo_pi`, is a single function that runs the established simulation for a given sample size `n` and returns just the final estimate for that `n`. The next custom function, `monte_carlo_convergence`, takes the same approach that `monte_carlo_pi` does, but it utilizes the `np.cumsum` function on the array to build a running total of every step the simulation goes through. It then divides by the sample count at each step to get an estimate of the full samples. This is the convergence of the simulation towards the

actual value of π, defined by `np.pi`. The final custom function established is `run_multiple_simulations`, which calls `monte_carlo_pi` for a determined number of times and recording `abs(estimate - np.pi)`, returning all the errors and computing the standard deviation and mean. Finally, efficiency analysis of the Monte Carlo simulation was conducted through an application of the formula across the entire estimates array, and isolating the first index where a correct digit was reached using `np.argmax`. The corresponding sample count is then found, and it is repeated 50 times to take a median of the data. From all of this, graphs are then created plotting multiple different data points, such as the accuracy, a scatter graph showing the points inside the circle, and a convergence graph.

A flowchart of the entire program can be seen in Figure 1.

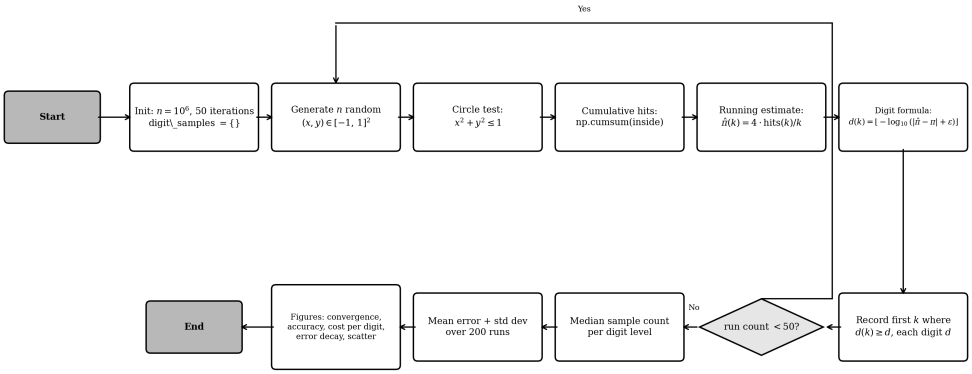


Figure 1: Flowchart of Python Program

5. Results

The results of the Monte Carlo simulation loop shows that at sample n = 1, the estimate for π is 4. This is the largest error in the estimation process because as the sample size increases, the simulation converges toward the true value of π (around 100,000 samples). This can be seen in Table 1

After running the program 200 times, we received a mean error of $\approx$ 0.0044 and a standard deviation of $\approx$ 0.0034 at 100,000 samples. Both of these small values for mean and standard deviation over a large sample size shows that the simulation is working correctly and converging towards π. From the efficiency calculation, we get Table 2, which shows that as the digit

Table 1: Estimates of π for increasing sample sizes.

Samples	**Estimate**
1	4.00000
10	3.60000
100	3.08000
1000	3.21200
10000	3.13800
100000	3.13344

accuracy increases, the ratio increases about 10x compared to the previous digit, as the ratio is measured in comparison to the digit that came before it. For example, 2 digits of accuracy is $\approx 6x$ more intensive to compute than 1 digit, which itself is $\approx 7x$ more intensive to compute than the whole number.

Table 2: Efficiency ratio for π at each digit.

Digit	**Median**	**Ratio of Previous**
0	1.0	1.0x
1	7.0	7.0x
2	44.5	6.357142857142857x
3	368.0	8.269662921348315x
4	925.0	2.5135869565217392x
5	5615.0	6.07027027027027x

A visual representation of the convergence of the simulation, which can be seen in Figure 2, confirms that as the simulation runs, it approaches π.

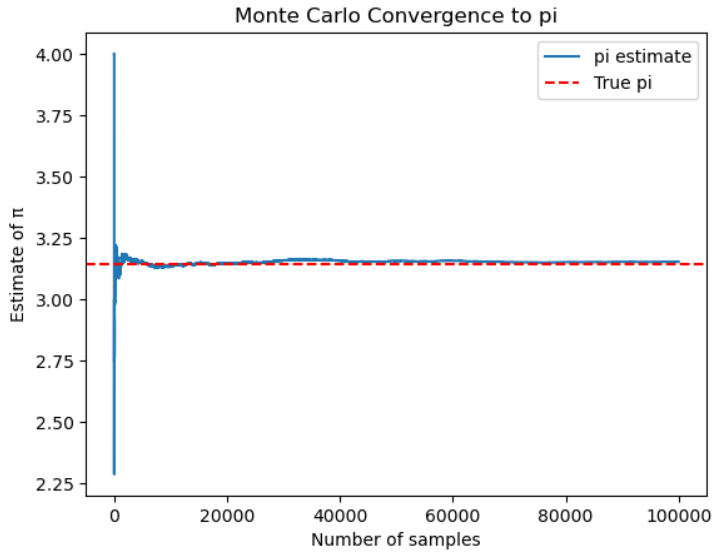


Figure 2: Convergence graph of Monte Carlo simulation.

After around 10,000 samples, as can be seen in the figure, the accuracy of the simulation settles around π, with no additional increases in accuracy with our sample size, which follows the established theory.

References

[1] R. Blankman, A history of pi: Global calculations, HMH Education Company (2021).

[2] Exploratorium, A brief history of pi (2024).

[3] W. Kenton, Monte carlo simulation: What it is, how it works, history, 4 key steps (2026).

[4] IBM, What is monte carlo simulation? (2025).

[5] M. Newman, Computational Physics, 2nd Edition, Independently Published, 2025.